

Báo cáo bài tập về nhà - Bài dễ nhất

Nhóm thực hiện: Nhóm 7

Danh sách thành viên:

- Vũ Minh Phương
- Võ Lê Ngọc Thịnh

1. Mô tả bài toán

Bài toán cho một bản đồ hình chữ nhật kích thước $W \times H$, trên đó có n vùng theo dõi Aura được biểu diễn bởi các hình chữ nhật có thể lồng vào nhau nhưng không giao cắt và m Hunter xuất hiện tại các vị trí xác định kèm theo lượng Aura tương ứng. Mỗi Hunter chỉ được ghi nhận bởi vùng theo dõi nhỏ nhất chứa vị trí của họ; nếu không thuộc bất kỳ vùng nào, lượng Aura của họ được xem là “không được theo dõi”. Yêu cầu của bài toán là tính tổng lượng Aura được ghi nhận trong từng vùng và tổng lượng Aura không được theo dõi.

2. Hướng tiếp cận bài toán

Ta sẽ tiếp cận bài toán này theo từng subtask. Mỗi subtask sẽ xây dựng dựa trên những hạn chế của bước trước.

- Subtask 1 ($n, m \leq 10^3$): Giải pháp Brute Force
- Subtask 2 ($W, H \leq 10^3$): Xây dựng Segment Tree 2D mô phỏng lại mặt phẳng tọa độ
- Subtask 3 (Không ràng buộc): Sử dụng kĩ thuật Sweep Line

3. Subtask 1 ($n, m \leq 10^3$)

3.1. Ý tưởng & Phương pháp

Ở subtask nhỏ này, số lượng vùng và số lượng Hunter đều chỉ đến 10^3 , do đó ta có thể sử dụng phương pháp **Brute Force**.

Cụ thể, với mỗi Hunter, ta duyệt qua toàn bộ n vùng, kiểm tra xem Hunter nằm trong vùng nào và chọn vùng có diện tích nhỏ nhất.

3.2. Mô tả thuật toán

1. Đối với mỗi Hunter j , lần lượt kiểm tra toàn bộ n vùng theo dõi.
2. Ghi nhận tất cả các vùng mà vị trí của Hunter j nằm bên trong.
3. Nếu không có vùng nào chứa Hunter, cộng lượng Aura t_j vào nhóm “không được theo dõi”.
4. Ngược lại, chọn vùng có diện tích nhỏ nhất trong số các vùng đã ghi nhận và cộng thêm Aura của Hunter vào tổng Aura của vùng đó.

3.3. Lý do lựa chọn phương pháp

- Số lượng phép kiểm tra tối đa chỉ là $n \cdot m = 10^6$, hoàn toàn chấp nhận được.
- Không cần cấu trúc dữ liệu phức tạp, ít rủi ro, triển khai đơn giản.
- Mục tiêu của Subtask 1 là kiểm tra khả năng nhận diện bài toán và dùng phương pháp cơ bản nhất. Và có thể dùng thuật toán ở Subtask 1 để kiểm thử các thuật toán xây dựng cho Subtask 2 & 3

3.4. Độ phức tạp thuật toán

Độ phức tạp thời gian

- Kiểm tra quan hệ Hunter $\in$ Vùng: $O(1)$.
- Với m Hunter và n vùng, tổng thời gian: $O(nm)$

Độ phức tạp không gian

- Không gian chỉ dùng mảng lưu vùng và Hunter: $O(n + m)$

4. Subtask 2 ($W, H \leq 10^3$)

4.1. Ý tưởng và phương pháp thiết kế

Vì $W, H \leq 10^3$, toàn bộ mặt phẳng lưới kích thước tối đa chỉ có 10^6 ô. Mục tiêu của Subtask 2 là xác định vùng theo dõi nhỏ nhất bao phủ mỗi điểm có Hunter. Một cách tiếp cận hiệu quả là sử dụng **Segment Tree 2D** để thực hiện truy vấn “điểm thuộc vùng chữ nhật nào” trong thời gian nhanh hơn $O(n)$.

Ý tưởng chính gồm ba bước:

1. **Sắp xếp vùng theo diện tích giảm dần** và gán mỗi vùng một `id`.
2. **Duyệt lần lượt từ vùng lớn đến vùng nhỏ**, cập nhật vùng đó vào một Segment Tree 2D bằng phép *range-assign max*.
3. Khi xử lý Hunter, chỉ cần thực hiện truy vấn 1 điểm trên cây 2D để tìm ra vùng có `id` lớn nhất bao phủ điểm đó $\rightarrow$ đó chính là vùng nhỏ nhất (do ta đã sắp xếp ngược).

Phương pháp này thuộc nhóm **Transform and Conquer**: biến bài toán hình học về một bài toán quản lý hình chữ nhật bằng cây phân đoạn hai chiều.

4.2. Tại sao phương pháp này phù hợp

- Vì $W, H \leq 10^3$, kích thước cây Segment Tree 2D ($\sim 4W \times 4H$) vẫn nằm trong mức cho phép bộ nhớ.
- Vùng có thể **lồng nhau**, nên ta không thể dùng kỹ thuật tô lưới tuyến tính theo vùng. Segment Tree 2D khắc phục được nhược điểm này.
- Update theo vùng và query theo điểm đều là các thao tác tự nhiên trên Segment Tree 2D.
- Sắp xếp vùng theo diện tích giảm dần đảm bảo rằng: Vùng lớn cập nhật trước $\rightarrow$ vùng nhỏ cập nhật sau $\rightarrow$ vùng nhỏ “ghi đè” vùng lớn khi Hunter truy vấn.

Nhờ vậy, truy vấn chỉ điểm cho ra chính xác vùng nhỏ nhất chứa điểm.

4.3. Mô tả chi tiết thuật toán

1. Đọc toàn bộ vùng và tính diện tích từng vùng:

$$area_i = (A_i - X_i) \cdot (B_i - Y_i)$$

2. Sắp xếp vùng theo diện tích giảm dần. Vùng lớn nhất đứng trước, vùng nhỏ nhất đứng cuối. Gán id bằng chỉ số của vùng này trên mảng đã được sắp xếp.
3. Khởi tạo cây Segment Tree 2D để quản lý một vùng chữ nhật lớn $W * H$, với giá trị mặc định = 0 (không có vùng nào).
4. Cập nhật cây với từng vùng bằng thao tác gán max.
5. Xử lý từng Hunter bằng cách truy giá trị của một ô và giá trị này chính là vị trí của vùng theo dõi trên mảng đã sắp xếp, còn nếu giá trị bằng 0 thì ô này nằm ngoài vùng được theo dõi.

4.4. Độ phức tạp thuật toán

Thời gian

- Khởi tạo Segment Tree 2D: $O(W \cdot H)$
- Cập nhật một vùng theo dõi: $O(\log W \cdot \log H)$
- Tổng cập nhật cho n vùng: $O(n \log W \log H)$
- Truy vấn vị trí của một Hunter: $O(\log W \log H)$
- Tổng truy vấn cho m Hunter: $O(m \log W \log H)$
- **Tổng thời gian toàn thuật toán:**

$$O((n + m) \log W \log H)$$

Không gian

- Bộ nhớ cho Segment Tree 2D: $O(WH)$
- Lưu trữ vùng và Hunter: $O(n + m)$
- **Tổng không gian:**

$$O(WH)$$

5.Sweep-line

Khi các phương pháp duyệt thô (Brute-Force) không còn đáp ứng được yêu cầu về thời gian trong những bài toán hình học hai chiều, kỹ thuật Sweep-line trở thành một giải pháp mạnh nhờ khả năng giảm chiều và tối ưu hóa đáng kể quá trình xử lý.

Sweep-line là một ví dụ kinh điển của chiến lược thiết kế thuật toán Transform and Conquer: thay vì làm việc trực tiếp trên không gian 2D, ta biến đổi bài toán về dạng 1D đơn giản hơn để xử lý hiệu quả.

5.1. Nguyên lý cốt lõi

Sweep-line hoạt động theo hai giai đoạn tách biệt:

Giai đoạn Transform:

Thay vì thao tác trực tiếp với các đối tượng 2D, ta chuyển đổi chúng thành một tập các “sự kiện” 1D. Cụ thể, mỗi hình chữ nhật hoặc mỗi điểm được ánh xạ thành các sự kiện xảy ra tại một giá trị tọa độ cố định (thường là trục x). Ví dụ, một hình chữ nhật tạo ra hai sự kiện: mở (OPEN) tại cạnh trái và đóng (CLOSE) tại cạnh phải; một Hunter tạo ra sự kiện truy vấn (QUERY) tại vị trí của nó. Các sự kiện này được gom vào một danh sách và sắp xếp theo thứ tự tăng dần của x .

Giai đoạn Conquer:

Sau khi đã chuyển tất cả dữ liệu 2D về chuỗi sự kiện 1D, ta mô phỏng một đường quét (sweep line) di chuyển dọc trục x . Tại mỗi sự kiện, bài toán 2D tương ứng sẽ được rút gọn thành một bài toán 1D trên trục y , chẳng hạn như thêm một khoảng, xóa một khoảng hoặc truy vấn một điểm. Để hỗ trợ các thao tác này, cần có một cấu trúc dữ liệu duy trì trạng thái của các vùng đang hoạt động theo trục y , chẳng hạn một Segment Tree, Interval Tree hoặc Balanced BST. Cấu trúc này cho phép thêm/xóa vùng nhanh chóng và trả lời truy vấn Hunter trong thời gian logarit.

5.2. Các thành phần chính của mô hình Sweep-line

Kỹ thuật Sweep-line trong các bài toán xử lý hình chữ nhật thường gồm hai thành phần chủ đạo:

1. Danh sách sự kiện:

Đây là toàn bộ các sự kiện OPEN, CLOSE và QUERY được sắp xếp theo trục x . Danh sách này là kết quả của giai đoạn Transform và đóng vai trò là “mặt phẳng thời gian” cho đường quét.

2. Cấu trúc dữ liệu trạng thái:

Đây là bộ phận cốt lõi của giai đoạn Conquer. Nó lưu trữ thông tin về các vùng đang hoạt động tại thời điểm đường quét đứng yên, đồng thời hỗ trợ ba thao tác:

- thêm vùng mới (OPEN),
- xóa vùng đã kết thúc (CLOSE),
- truy vấn vùng chứa một điểm theo trục y (QUERY).

Tùy theo yêu cầu bài toán, cấu trúc trạng thái có thể là cây cân bằng, Segment Tree, Fenwick Tree hoặc các cấu trúc khoảng theo thứ tự ưu tiên.

Sweep-line cho phép giảm bài toán từ xử lý hình chữ nhật 2D phức tạp xuống chuỗi thao tác 1D trên các cấu trúc dữ liệu chuyên dụng, nhờ đó đạt hiệu suất vượt trội và xử lý tốt các ràng buộc lớn trong Subtask 3.

6. Subtask 3 — Giải bằng Sweep-line (Transform and Conquer)

6.1. Ý tưởng và phương pháp thiết kế

Với ràng buộc lớn ($W, H \leq 10^9$ và $n, m \leq 10^5$), các phương pháp brute-force hoặc lưới 2D đều không khả thi. Do các vùng theo dõi là hình chữ nhật không giao nhau nhưng có thể lồng nhau, ta cần một phương pháp xử lý hình học 2D hiệu quả. Kỹ thuật Sweep-line là lựa chọn phù hợp vì nó cho phép biến đổi bài toán 2D thành bài toán 1D bằng cách quét theo trục x , đồng thời dùng một cấu trúc dữ liệu duy trì trạng thái theo trục y để truy vấn nhanh.

Phương pháp này thuộc nhóm **Transform and Conquer**: thay vì xử lý trực tiếp trên mặt phẳng hai chiều, ta biến đổi toàn bộ hình chữ nhật thành một chuỗi sự kiện 1D và xử lý chúng theo thứ tự.

6.2. Tại sao phương pháp này phù hợp

- Chiều cao bản đồ rất lớn (tối đa 10^9), không thể dùng lưới hay Segment Tree 2D.

- Các vùng không giao nhau giúp trạng thái Sweep-line gọn hơn, vì cùng thời điểm chỉ một số ít vùng nằm trong tập “đang hoạt động”.
- Sweep-line xử lý hiệu quả các bài toán dạng “điểm thuộc hình chữ nhật nào” bằng cách:
 - Xử lý hình chữ nhật theo sự kiện (OPEN/CLOSE),
 - Và truy vấn điểm đúng tại thời điểm đường quét đi ngang qua.
- Thuật toán đạt độ phức tạp xấp xỉ

$$O((n + m) \log(n + m))$$

phù hợp với giới hạn 10^5 .

6.3. Chi tiết thuật toán

Thuật toán gồm ba giai đoạn chính:

(1) Tạo sự kiện (Transform)

Mỗi vùng tạo ra hai sự kiện:

- **OPEN** tại $x = x_1$ (bắt đầu ảnh hưởng của vùng)
- **CLOSE** tại $x = x_2$ (kết thúc ảnh hưởng của vùng)

Mỗi Hunter tạo ra một sự kiện:

- **QUERY** tại $x = C_j$ (cần xác định Hunter thuộc vùng nào)

Mỗi sự kiện có dạng: $(x, \text{type}, y1, y2, \text{id})$, trong đó:

- $\text{type} = +1 \rightarrow \text{OPEN}$
- $\text{type} = -1 \rightarrow \text{CLOSE}$
- $\text{type} = 0 \rightarrow \text{QUERY}$ (khi đó $y1$ là tọa độ y của Hunter, $y2$ là lượng aura)

Tất cả sự kiện được gom lại vào một mảng duy nhất.

(2) Rời rạc hóa tọa độ Y

Vì H rất lớn, ta không thể tạo segment tree dài 10^9 .

Thay vào đó, ta thu thập tất cả các tọa độ biên y_1, y_2 của vùng và y của Hunter, sau đó:

- Sort
- Loại trùng
- Map về chỉ số trong mảng

Điều này cho phép xây dựng cây Segment Tree theo trục y có kích thước nhỏ hơn rất nhiều.

(3) Quét theo trục X (Conquer) + duy trì trạng thái theo Y

Sắp xếp tất cả sự kiện theo:

- Tăng dần theo x
- Với cùng x , thứ tự: OPEN $\rightarrow$ QUERY $\rightarrow$ CLOSE (đảm bảo rằng vùng mở ra tại x sẽ được tính cho Hunter đứng tại x)

Ta duyệt từng sự kiện theo thứ tự:

• OPEN (type = +1)

Cập nhật đoạn $[y_1, y_2]$ trong Segment Tree để gán giá trị = id vùng.

Segment tree lưu vùng đang hoạt động hiện tại, cho phép truy vấn theo điểm.

• CLOSE (type = -1)

Cập nhật đoạn $[y_1, y_2]$ để xóa ảnh hưởng của vùng (gán lại -1).

• QUERY Hunter (type = 0)

Truy vấn điểm y để tìm vùng đang hoạt động bao phủ Hunter.

- Nếu Segment Tree trả về $id = -1$: Hunter **không thuộc vùng nào**
- Nếu $id \geq 0$: Hunter thuộc vùng đó, tăng lượng auro được theo dõi bởi vùng đó lên.

6.4. Độ phức tạp thuật toán

Thời gian

- Rời rạc hóa trục y : $O((n + m) \log(n + m))$
- Sắp xếp toàn bộ sự kiện: $O((n + m) \log(n + m))$
- Mỗi sự kiện xử lý một update/query trên segment tree: $O(\log(n + m))$
- **Tổng thời gian toàn thuật toán:** $O((n + m) \log(n + m))$

Không gian

- Mảng rời rạc hóa: $O(n + m)$
- Segment Tree theo trục y : $O(n + m)$
- Lưu sự kiện: $O(n + m)$
- **Tổng độ phức tạp không gian:** $O(n + m)$

6.5. Code tham khảo

```
import sys
from bisect import bisect_left

input = sys.stdin.readline

W, H, R, Q = map(int, input().split())

events = []
ys = []

for idx in range(R):
    x1, y1, x2, y2 = map(int, input().split())
    events.append((x1, 1, y1, y2, idx))
    events.append((x2, -1, y1, y2, idx))
    ys.extend((y1, y2))

region_aura = [0] * R

for _ in range(Q):
    x, y, a = map(int, input().split())
    events.append((x, 0, y, a, -1))
    ys.append(y)

ys_sorted = sorted(set(ys))
y_id = {v: i for i, v in enumerate(ys_sorted)} # Fast mapping
S = len(ys_sorted)

def getY(v): return y_id[v]

seg = [-1] * (2 * S)

def seg_update(l, r, val):
    l += S
    r += S
    while l <= r:
        if l & 1:
            seg[l] = val
            l += 1
        if not r & 1:
            seg[r] = val
            r -= 1
        l >>= 1
        r >>= 1

def seg_get(p):
    p += S
    res = -1
```

```
while p:
    if seg[p] != -1:
        res = seg[p]
    p >>= 1
return res
```

```
events.sort(key=lambda x: (x[0], -x[1]))
```

```
free = 0
```

```
for x, typ, a, b, idx in events:
    if typ == 1 or typ == -1:
        l = getY(a)
        r = getY(b)
        seg_update(l, r, idx if typ == 1 else -1)
    else:
        hy = getY(a)
        rid = seg_get(hy)
        if rid == -1:
            free += b
        else:
            region_aura[rid] += b
```

```
print(*region_aura, free)
```